



Jay Alammam

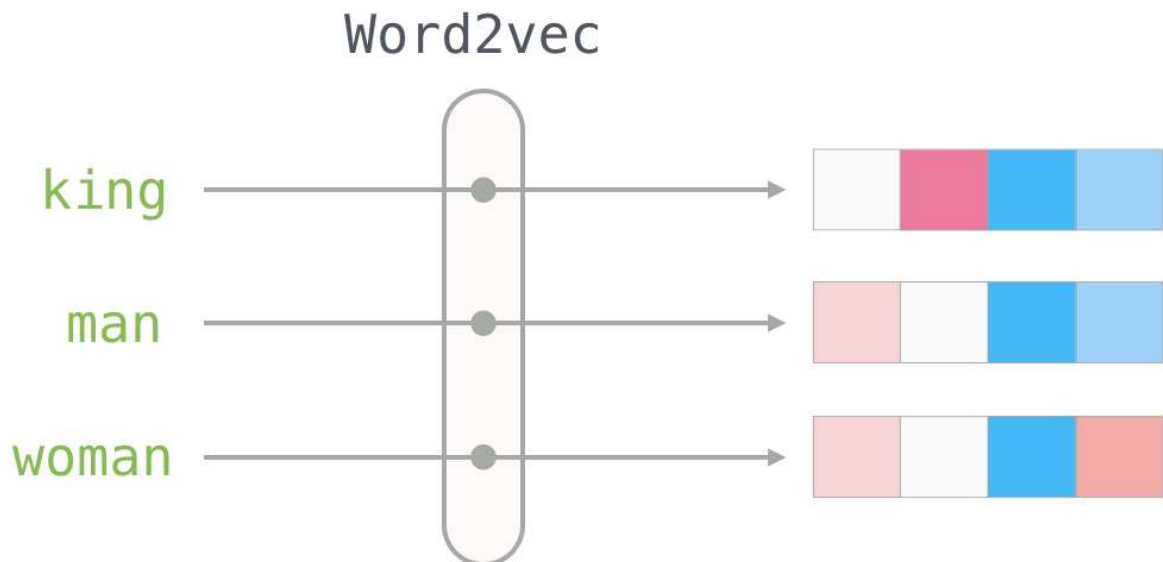
Visualizing machine learning one concept at a time.

Read our book, [Hands-On Large Language Models](#) and follow me on [LinkedIn](#), [Bluesky](#), [Substack](#), [X](#), [YouTube](#)

The Illustrated Word2vec

[Blog](#) [About](#)

Discussions: [Hacker News](#) (347 points, 37 comments), [Reddit r/MachineLearning](#) (151 points, 19 comments)
Translations: [Chinese \(Simplified\)](#), [French](#), [Korean](#), [Portuguese](#), [Russian](#)



"There is in all things a pattern that is part of our universe. It has symmetry, elegance, and grace - those qualities you find always in that which the true artist captures. You can find it in the turning of the seasons, in the way sand trails along a ridge, in the branch clusters of the creosote bush or the pattern of its leaves.

We try to copy these patterns in our lives and our society, seeking the rhythms, the dances, the forms that comfort. Yet, it is possible to see peril in the finding of ultimate perfection. It is clear that the ultimate pattern contains its own fixity. In such perfection, all things move toward death." ~ Dune (1965)

I find the concept of embeddings to be one of the most fascinating ideas in machine learning. If you've ever used Siri, Google Assistant, Alexa, Google Translate, or even smartphone keyboard with next-word prediction, then chances are you've benefitted from this idea that has become central to Natural Language Processing models. There has been quite a development over the last couple of decades in using embeddings for neural models (Recent developments include contextualized word embeddings leading to cutting-edge models like [BERT](#) and [GPT2](#)).

Word2vec is a method to efficiently create word embeddings and has been around since 2013. But in addition to its utility as a word-embedding method, some of its concepts have been shown to be effective in creating recommendation engines and making sense of sequential data even in commercial, non-language tasks. Companies like [Airbnb](#), [Alibaba](#), [Spotify](#), and [Anghami](#) have all benefitted from carving out this brilliant piece of machinery from the world of NLP and using it in production to empower a new breed of recommendation engines.

In this post, we'll go over the concept of embedding, and the mechanics of generating embeddings with word2vec. But let's start with an example to get familiar with using vectors to represent things. Did you know that a list of five numbers (a vector) can represent so much about your personality?

Personality Embeddings: What are you like?

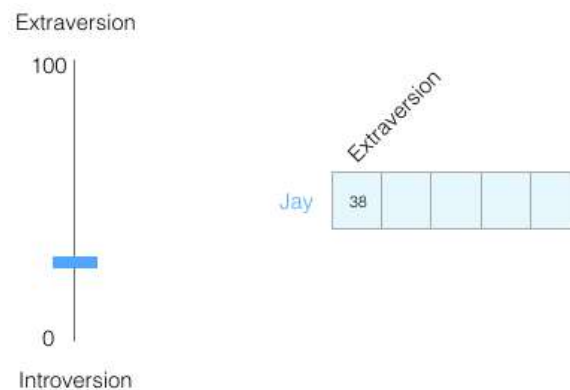
"I give you the desert chameleon, whose ability to blend itself into the background tells you all you need to know about the roots of ecology and the foundations of a personal identity" ~Children of Dune

On a scale of 0 to 100, how introverted/extraverted are you (where 0 is the most introverted, and 100 is the most extraverted)? Have you ever taken a personality test like MBTI – or even better, the [Big Five Personality Traits](#) test? If you haven't, these are tests that ask you a list of questions, then score you on a number of axes, introversion/extraversion being one of them.

Openness to experience	79 out of 100
Agreeableness	75 out of 100
Conscientiousness	42 out of 100
Negative emotionality	50 out of 100
Extraversion	58 out of 100

Example of the result of a Big Five Personality Trait test. It can really tell you a lot about yourself and is shown to have predictive ability in [academic](#), [personal](#), and [professional success](#). This is one place to find your results.

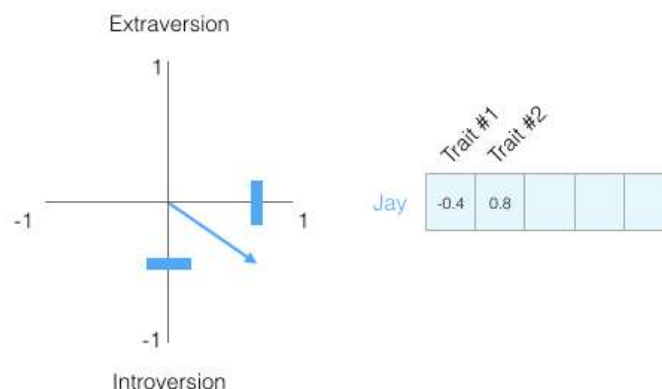
Imagine I've scored 38/100 as my introversion/extraversion score. we can plot that in this way:



Let's switch the range to be from -1 to 1:



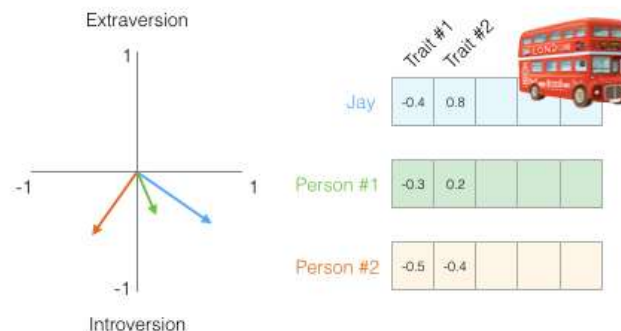
How well do you feel you know a person knowing only this one piece of information about them? Not much. People are complex. So let's add another dimension – the score of one other trait from the test.



We can represent the two dimensions as a point on the graph, or better yet, as a vector from the origin to that point. We have incredible tools to deal with vectors that will come in handy very shortly.

I've hidden which traits we're plotting just so you get used to not knowing what each dimension represents – but still getting a lot of value from the vector representation of a person's personality.

We can now say that this vector partially represents my personality. The usefulness of such representation comes when you want to compare two other people to me. Say I get hit by a **bus** and I need to be replaced by someone with a similar personality. In the following figure, which of the two people is more similar to me?



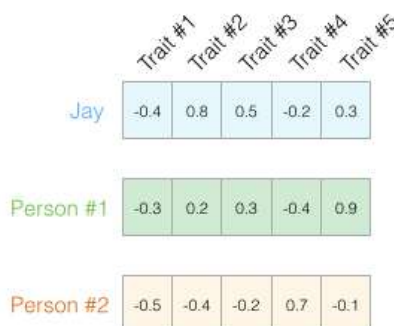
When dealing with vectors, a common way to calculate a similarity score is `cosine_similarity`:

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#1}) = 0.87 \quad \checkmark$$

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#2}) = -0.20$$

Person #1 is more similar to me in personality. Vectors pointing at the same direction (length plays a role as well) have a higher cosine similarity score.

Yet again, two dimensions aren't enough to capture enough information about how different people are. Decades of psychology research have led to five major traits (and plenty of sub-traits). So let's use all five dimensions in our comparison:



The problem with five dimensions is that we lose the ability to draw neat little arrows in two dimensions. This is a common challenge in machine learning where we often have to think in higher-dimensional space. The good thing is, though, that `cosine_similarity` still works. It works with any number of dimensions:

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#1}) = 0.66 \quad \checkmark$$

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#2}) = -0.37$$

`cosine_similarity` works for any number of dimensions. These are much better scores because they're calculated based on a higher resolution representation of the things being compared.

At the end of this section, I want us to come out with two central ideas:

1. We can represent people (and things) as vectors of numbers (which is great for machines!).
2. We can easily calculate how similar vectors are to each other.

1- We can represent things (and people) as vectors of numbers (Which is great for machines!)

Jay	-0.4	0.8	0.5	-0.2	0.3
-----	------	-----	-----	------	-----

2- We can easily calculate how similar vectors are to each other

The people most similar to Jay are:

cosine_similarity ▼

Person #1	0.86
Person #2	0.5
Person #3	-0.20

Word Embeddings

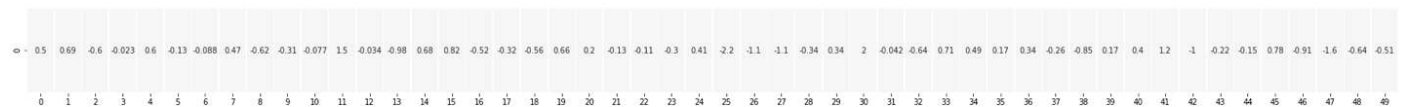
“The gift of words is the gift of deception and illusion” ~Children of Dune

With this understanding, we can proceed to look at trained word-vector examples (also called word embeddings) and start looking at some of their interesting properties.

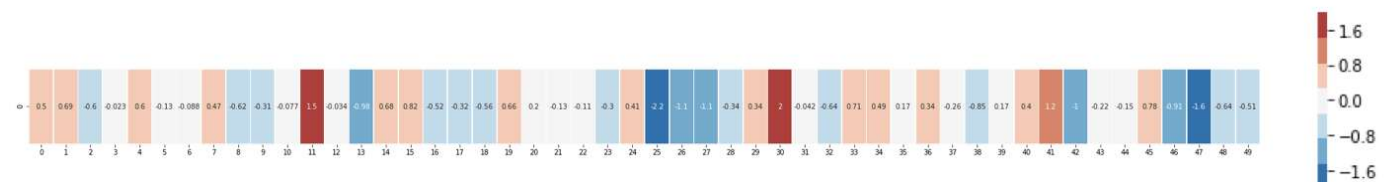
This is a word embedding for the word “king” (GloVe vector trained on Wikipedia):

```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,  
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961  
 , -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,  
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,  
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```

It's a list of 50 numbers. We can't tell much by looking at the values. But let's visualize it a bit so we can compare it other word vectors. Let's put all these numbers in one row:

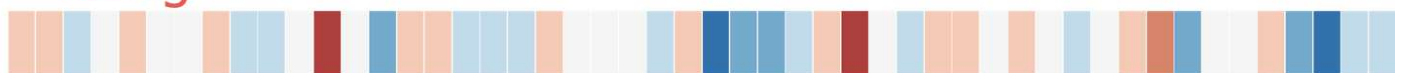


Let's color code the cells based on their values (red if they're close to 2, white if they're close to 0, blue if they're close to -2):

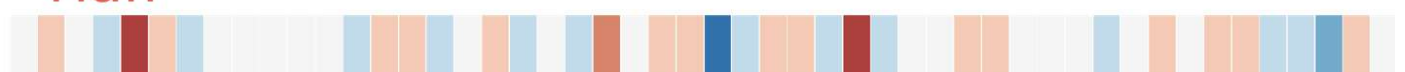


We'll proceed by ignoring the numbers and only looking at the colors to indicate the values of the cells. Let's now contrast “King” against other words:

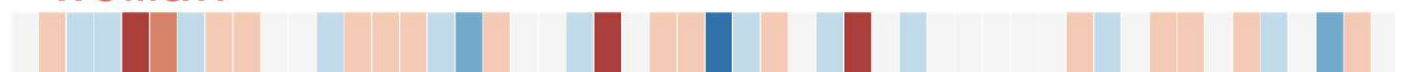
“king”



“Man”

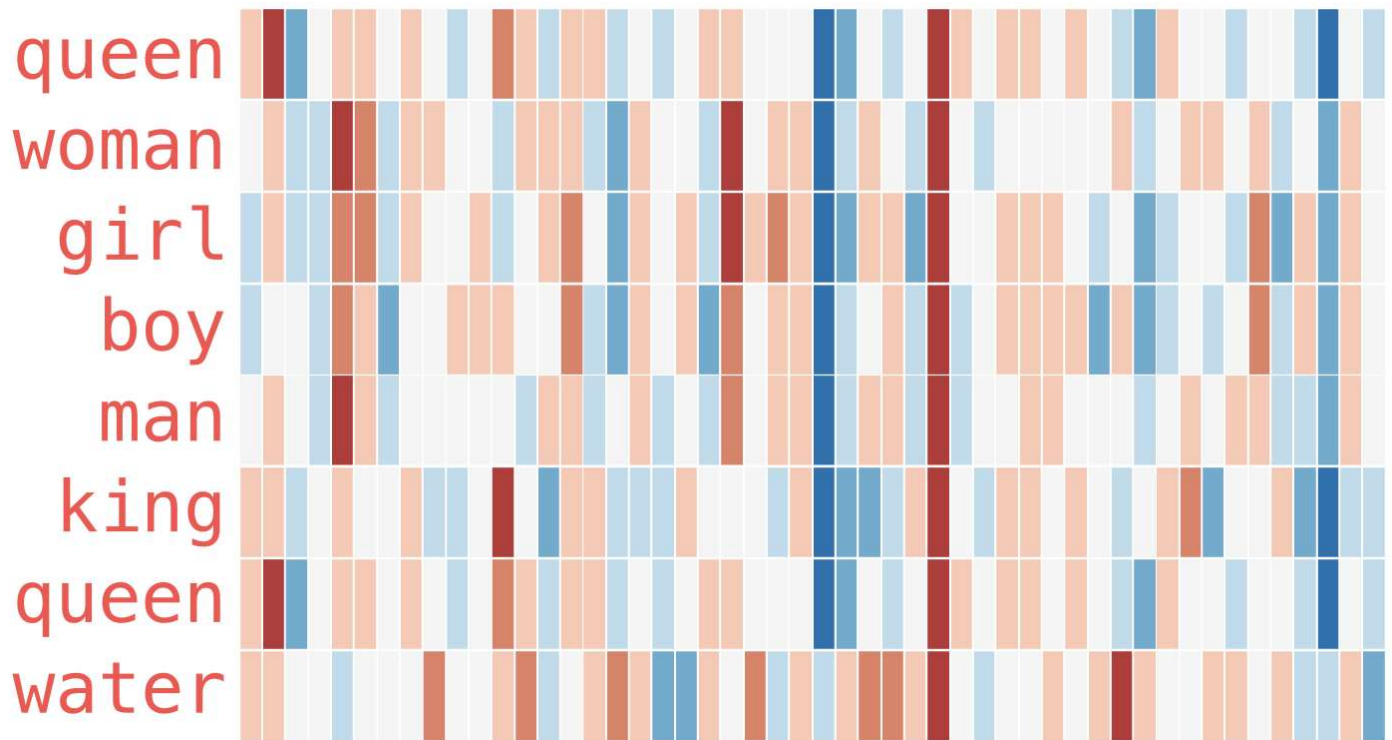


“Woman”



See how “Man” and “Woman” are much more similar to each other than either of them is to “king”? This tells you something. These vector representations capture quite a bit of the information/meaning/associations of these words.

Here's another list of examples (compare by vertically scanning the columns looking for columns with similar colors):



A few things to point out:

1. There's a straight red column through all of these different words. They're similar along that dimension (and we don't know what each dimension codes for)
2. You can see how "woman" and "girl" are similar to each other in a lot of places. The same with "man" and "boy"
3. "boy" and "girl" also have places where they are similar to each other, but different from "woman" or "man". Could these be coding for a vague conception of youth? possible.
4. All but the last word are words representing people. I added an object (water) to show the differences between categories. You can, for example, see that blue column going all the way down and stopping before the embedding for "water".
5. There are clear places where "king" and "queen" are similar to each other and distinct from all the others. Could these be coding for a vague concept of royalty?